CUDAsearch: Distributed GPU-Accelerated Dense Vector Similarity Search

1. Project Focus

We propose to build **CUDAsearch**, a distributed dense vector similarity search system for retrieval-style workloads. Given a large matrix of pre-computed embeddings and a query embedding, the system returns the top-k most similar vectors using maximum inner product search (MIPS), that is finding the database vector whose dot product with the query is largest (which corresponds to cosine similarity when vectors are normalized). This primitive appears in semantic search, retrieval-augmented generation (RAG) [1], and recommendation systems.

Parallelism is essential because exhaustive similarity search requires computing one dot product per database vector. For a database with millions of vectors, this becomes a large matrix-vector multiplication followed by a top-k selection. GPUs are well-suited for the compute-intensive inner-product stage, while MPI enables the database to be sharded across multiple GPUs and nodes. This project therefore fits the goal of combining CUDA and distributed-memory parallelism in one application.

2. Core Algorithms

- **Dense similarity computation.** The main computation is a matrix-vector product where each row of the embedding matrix is compared against the query vector via an inner product. Embeddings are pre-computed offline using a pretrained sentence encoder [2].
- **Top-k selection.** After computing similarity scores, each process keeps its local top-k candidates (using a heap or partial sort).
- **Distributed top-k merge.** Each MPI rank stores a shard of the database, computes its local top-k, and sends these candidates to rank 0, which merges them into the global top-k result.
- **CPU reference baseline.** We will implement a simple CPU baseline for correctness checking and speedup measurements, comparable to a FAISS flat index [3].

3. Parallel Computing Approach

CUDA.

- We will write a CUDA kernel from scratch for the similarity computation. A baseline version will assign threads to rows of the embedding matrix and accumulate dot products in parallel.
- Our main non-trivial optimization will be **tiling with shared memory** [4]: blocks of the query vector and/or matrix data will be staged through shared memory to reduce global memory traffic and improve memory access efficiency.
- As a second optimization, we will explore **INT8 quantization** of the embedding matrix . We will measure the speedup against the FP32 baseline while tracking any recall@k degradation.
- We will analyze thread/block organization, register usage, and memory hierarchy behavior, and we will measure the effect of coalesced memory access on performance.

MPI.

- The embedding database will be partitioned row-wise across MPI ranks, with one GPU used per rank.

- For each query, rank 0 will broadcast the query vector to all ranks. Each rank will launch its CUDA kernel on its local shard, compute a local top-k, and send the local candidates back to rank 0.
- Rank 0 will merge the local results into the final global top-k. This design gives a simple and clean multi-GPU implementation that directly exposes both computation and communication costs.

OpenMP.

- **If useful and if time allows**, we will use OpenMP on the host side for auxiliary CPU work such as preprocessing inputs or parallelizing the final merge step. This is not central to the project but may provide a small additional comparison point.

4. Planned Deliverables

- A working C++/CUDA/MPI implementation of distributed dense vector similarity search, open-sourced on GitHub.
- One baseline CUDA kernel and one optimized CUDA kernel using shared-memory tiling, with quantitative comparison between them.
- Multi-GPU execution using MPI, including at least basic strong-scaling measurements across multiple GPUs.
- Profiling and benchmarking results, including kernel runtime, end-to-end query latency, and analysis of performance.
- A theoretical performance analysis including arithmetic intensity estimates and strong-scaling measurements across multiple GPUs.
- A comparison against a CPU baseline and a qualitative or small-scale quantitative comparison with FAISS [3].

5. Timeline & Risk Assessment

Milestone	Due	Planned Work
Design	Apr 27	Build CPU baseline, define memory layout, set up data loading, and implement a first serial top-k pipeline.
GPU Kernels	May 11	Implement baseline CUDA kernel, validate correctness, add profiling, and begin shared-memory tiling and INT8 optimization.
Distributed	May 27	Add MPI-based sharding across GPUs, implement distributed top-k aggregation, and collect scaling benchmarks.
Final Report	Jun 8	Finalize experiments, compare baseline vs. optimized versions, complete analysis, and write the 6-page report.

Identified risks.

- **Scope management:** there are many possible extensions for similarity search systems. To keep the project feasible, we focus on exhaustive search with one main CUDA optimization and a simple MPI decomposition.
- **Multi-GPU access and runtime environment:** access to multiple GPUs may affect the breadth of our scaling experiments. We will test the target environment and adapt benchmark

scope if needed.

- **Performance debugging:** GPU kernels can be correct but still slow because of poor memory access patterns. We mitigate this risk by first building a simple correct baseline and then introducing one optimization at a time.
- **Communication overhead:** for small queries or small database shards, MPI overhead may dominate. We will analyze this trade-off explicitly in the final report.

References

[1] P. Lewis et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] N. Reimers and I. Gurevych, Sentence-BERT: Sentence embeddings using Siamese BERT-networks, *Proceedings of EMNLP*, 2019.

[3] J. Johnson, M. Douze, and H. Jégou, Billion-scale similarity search with GPUs, *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[4] V. Volkov and J. W. Demmel, Benchmarking GPUs to tune dense linear algebra, *Proceedings of SC'08*, 2008.